

— 零训练 · 无任务头 · 开放词表

冻结的向量模型， 如何靠 **test-time compute** 学会给图片打标签

jina-embeddings-v5-omni-nano, 1B 检索模型, 权重全冻结

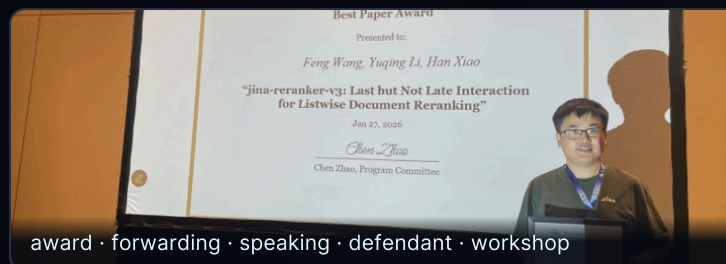
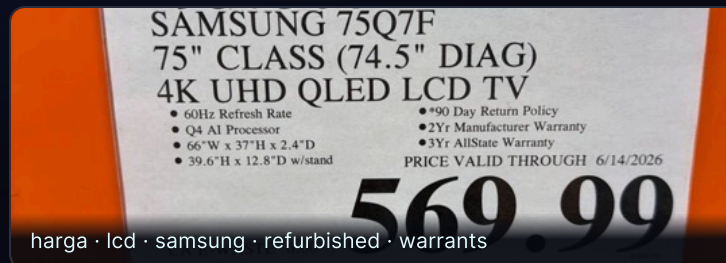
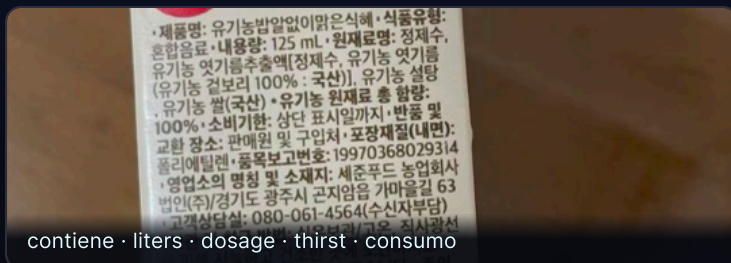
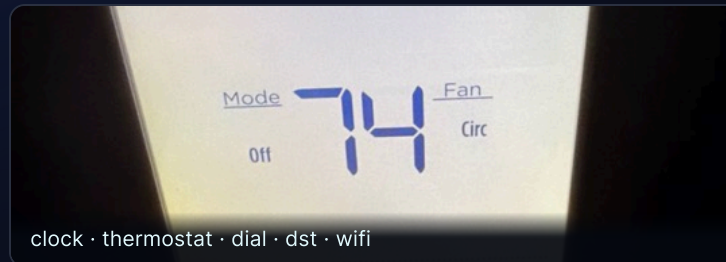
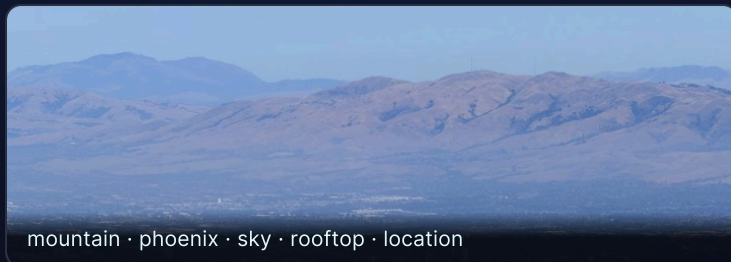
肖涵 Han Xiao

Elastic AI 副总裁 · Jina AI 创始人

@hxiao · hanxiao.io

— 先看结果

一个只会做检索的模型，直接吐出了标签。



没有训练、没有第二个模型、没有词典。标签全部来自推理期计算。

01 | 问题设定

给定一个冻结的检索编码器和一张图，允许的只有推理期计算

— SETUP

问题形式化。

- | **GIVEN** 一个向量模型 **jina-embeddings-v5-omni-nano** (~1B, 图文共享 768 维空间)，一张图。
- | **OUTPUT** 图中**每一个**物体的词：多标签、开放词表。
- | **FROZEN** 全部权重冻结。它是检索编码器，没有标注头、没有分类器，从未为此任务训练过。
- | **ALLOWED** 只允许在模型自身输出上做推理期计算，别的都不行。

不训练

不引入第二个模型

不用 WordNet / 词典

不用正则 / 词性标注

出现的任何标注能力，都必须来自**这个向量模型的 test-time compute**。

对向量模型来说，test-time compute 只有三种花法。

A

同一次前向，多做代数

不再只读池化向量，改读 **patch 级**输出，和 128k 词表逐一打分。计算量几乎不变。

B

新的视角，新的前向

把图切成多个 crop **重新编码**，小物体在自己的 crop 里占满画面。计算量线性增长。

C

对已有特征再加工

白化、最优传输、图传播、EM……在**同一批像素**上做更重的数学。看起来最「聪明」。

这次实验的结论提前说：**A 有效**，**B 有效**，**C 是幻觉**。

02 | 方法

词表即标签空间，patch 即候选区域，先验即背景

PIPELINE

四步，把检索器改造成标注器。

tokenizer 词表 → 标签空间



patch × 128,260 打分



减去背景先验 $\mu\ell$



词首 gate + embedding-NMS

01 词表就是开放标签空间

128,260 个 token 全部编码成文本向量。不需要人工标签集，模型认识多少词就有多少候选。

02 每个 patch 单独打分，再与全局向量融合

$0.7 \times \text{patch-max} + 0.3 \times \text{global}$ 。小物体不再被整图池化稀释——这是 mAP 从 0.264 到 0.635 的来源。

03 减去每个 token 自己的先验

高频 token 天然离所有图都近。减掉 $\mu\ell$ 之后，右尾才是真正的证据。

04 词首 gate 与向量空间 NMS

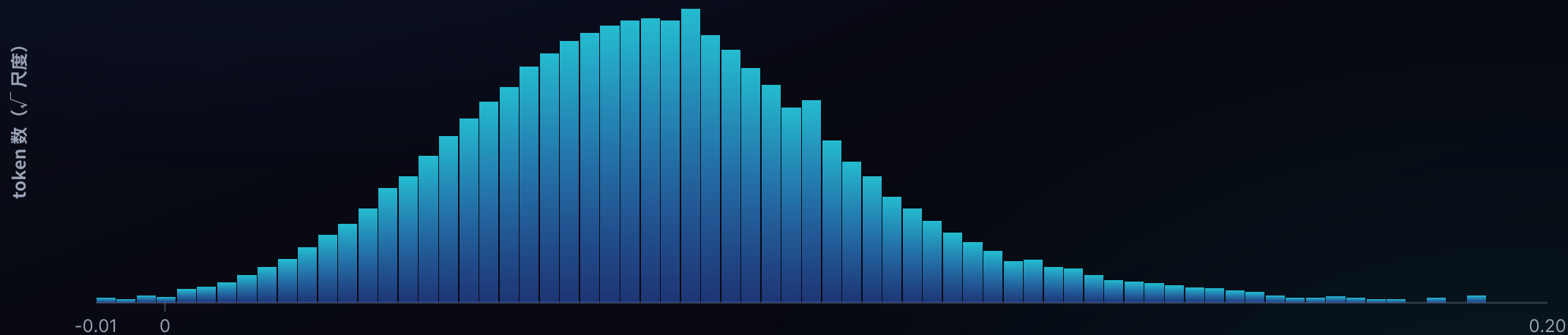
128,260 → 25,465 候选；再把 kitty/cat/kitten/chatte 这样的同义簇压成一个概念。

— 真实分布

12.8 万 token 的得分分布，一步步被整形。

1·原始融合得分

全部 128,260 个 token · 0.7 patch-max + 0.3 global



一条光滑的钟形：每个词都跟每张图「有点像」

03 | 结果

COCO-150, 80 类闭集评测：什么有用，什么是幻觉

COCO-150

四种做法的完整数字。

METHOD	P@1	P@3	R@5	MAP
global pooled	0.433	0.289	0.449	0.264
patch max + global	0.753	0.427	0.631	0.635
softpool	0.773	0.449	0.649	0.608
+ CWR multi-crop	0.813	0.476	0.680	0.710

全局池化 0.264 → patch 融合 0.635 → 多 crop 重编码 0.710。

三级台阶

精度是被「读得更多」和「看得更多」推上去的。



+0.371

A 类：同一次前向里改读 patch 级输出，几乎零额外算力

+0.075

B 类：额外 14 个 crop 重新编码，算力 $\times 15$

≤ 0

C 类：8 种再加工方法，全部持平或崩塌

全部杠杆

2024-2026 的免训练方法，我们挨个跑了一遍。



算力—精度

只有「新像素」能换到精度，「更重的数学」换不到。



为什么 C 类失败

这些方法的前提，在这里全都不成立。

×

白化 / GDA

前提是图文两个空间需要对齐。这里本来就是一个空间，白化只会破坏它。 $0.635 \rightarrow 0.060$ 。

×

EM-Dirichlet

前提是单标签、类间竞争。单纯形归一化直接杀死多标签。 $0.635 \rightarrow 0.170$ 。

×

ZLaP 图传播

图是建在弱的全局向量上的。基础错了，传播只会放大错误。 $0.635 \rightarrow 0.140$ 。

C 类不是「算得不够多」，是假设本身不成立。分数早已校准，没有额外信息可挖。

04 | 落地

同一套标签矩阵，跑在 macOS 原生应用 Omni 里

— OMNI

研究直接变成产品：本机全盘跨模态搜索。

~4%

标注开销：搭在原有 embedding 前向上顺带完成

0.847

HQ 重跑 (5-crop CWR) P@1, Swift/bf16 端口, 基线 0.773

32 帧

视频：每 240s 片段采样，跨空间与时间取 patch-max

图片

同一个标签矩阵，索引时顺带产出标签，无需二次前向。

视频

帧采样后共用同一套 patch 打分逻辑，时间维直接并入 max。

扫描版 PDF

页面当图处理，OCR 不可用的老档案也能被词检索到。

结论

检索就是 test-time compute。能力不在权重里，在你愿意为一次推理花多少算力上。

—— 但只有换来新信息的算力才算数

01 冻结的 1B 检索模型里，已经存在一个开放词表标注器

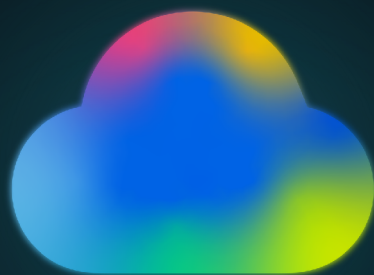
它不需要被训练出来，只需要被读出来。

02 A 与 B 有效，因为它们真的读进了新信息

A 读的是被池化丢掉的 patch，B 读的是被缩放丢掉的像素。

03 C 无效，因为它只是把同一批信息换个写法

没有新信息进来的算力，不管数学多漂亮，都换不到精度。



THANKS

肖涵 Han Xiao · Elastic AI 副总裁 · @hxiao

GITHUB REPO

